

COMSATS University Islamabad
Department of Bio-Sciences
Assignment # 02

Submission Date: 10 October 2026	Class: BSI – 5
Instructions: • Submit your work as a commented Python script (.py) or Jupyter notebook (.ipynb), exported to PDF/HTML, together with screenshots of every output. • Any copied content (from a classmate or an online source) will receive a score of zero. • Code that is submitted without evidence of understanding (e.g. unexplained AI-generated code, or an inability to explain it if asked) will be given a score of zero.	

Objective

The objective of this assignment is to perform Exploratory Data Analysis (EDA) on the **Air Quality** dataset using **Python**. You will use the **pandas**, **NumPy**, **SciPy**, and **Matplotlib/Seaborn** libraries to conduct descriptive statistical analysis, correlation analysis, grouped summaries, outlier detection, and visualizations to better understand the data.

Dataset

This assignment uses the same **Air Quality** dataset as the original R version — daily air-quality measurements recorded in New York, May to September 1973 — but you will load it directly in Python instead of R. The dataset is freely available online and can be loaded with either of the two methods below.

Option A — via the statsmodels package (recommended):

```
import statsmodels.api as sm

airquality = sm.datasets.get_rdataset('airquality', 'datasets', cache=True).data
airquality.head(6)
```

Option B — via pandas, reading a CSV copy of the dataset:

```
import pandas as pd

airquality = pd.read_csv('airquality.csv') # a CSV export of the dataset above
```

If you prefer Option B, first run Option A once, then save a local copy with `airquality.to_csv('airquality.csv', index=False)` — or search online for "R airquality dataset csv" and download any public mirror. Whichever source you use, confirm it has the same six columns listed below, and the same 153 rows, before proceeding.

The dataset contains the following columns:

- **Ozone:** Ozone concentration in parts per billion (ppb)
- **Solar.R:** Solar radiation in Langley's
- **Wind:** Average wind speed in miles per hour
- **Temp:** Temperature in degrees Fahrenheit
- **Month:** Month of the year (1–12)
- **Day:** Day of the month (1–31)

Tasks

Perform the following tasks on the Air Quality dataset in **Python**, using pandas / NumPy / SciPy / Matplotlib / Seaborn as appropriate.

1. Descriptive Statistics

- Load the dataset and display the first six rows (`df.head(6)`).
- Check the structure of the dataset — data types and dimensions (`df.info()`, `df.shape`, `df.dtypes`).
- Calculate and report the **mean** for the Ozone, Solar.R, Wind, and Temp columns (`df[cols].mean()`).
- Calculate and report the **median** for the same four columns (`df[cols].median()`).
- Calculate and report the **mode** for the same four columns. Pandas provides a built-in `.mode()`, but you must also write your own custom function that computes the mode manually (without using `.mode()` or `scipy.stats.mode`) and confirm your function's results match pandas' output.
- Calculate and report the **standard deviation** (`df[cols].std()`).
- Calculate and report the **variance** (`df[cols].var()`).
- Find the **range** (minimum and maximum) of each column (`df[cols].min()`, `df[cols].max()`).
- Calculate the **interquartile range (IQR)** for each column (`df[cols].quantile(0.75) - df[cols].quantile(0.25)`, or `scipy.stats.iqr`).
- Count the number of **missing values** in each column of the dataset (`df.isnull().sum()`).

2. Correlation Analysis

- Calculate the Pearson correlation coefficient between Ozone and Solar.R.
- Calculate the Pearson correlation coefficient between Ozone and Wind.
- Calculate the Pearson correlation coefficient between Ozone and Temp.
- Calculate the Pearson correlation coefficient between Solar.R and Wind.
- Calculate the Pearson correlation coefficient between Solar.R and Temp.
- Calculate the Pearson correlation coefficient between Wind and Temp.

Use `df['A'].corr(df['B'])`, or `scipy.stats.pearsonr(df['A'], df['B'])` after dropping rows where either column is missing with `.dropna()` — unlike pandas' `.corr()`, `pearsonr` does not handle NaN values automatically.

- Create a full **correlation matrix** for all numerical variables in the dataset (`df.corr(numeric_only=True)`), and visualize it as a heatmap using `seaborn.heatmap()`.
- Determine the strength of the correlation between the variables based on the coefficients calculated (use the scale: 0–0.1: No correlation, 0.1–0.3: Weak, 0.3–0.5: Moderate, 0.5–0.7: Strong, 0.7–1: Very strong).

3. Grouped Statistics

- Calculate the mean Ozone level for each month (`df.groupby('Month')['Ozone'].mean()`).
- Calculate the median Wind speed for each month.
- Calculate the maximum Temp for each month.
- Calculate the standard deviation of Solar.R for each month.
- Count the number of observations for each month in the dataset (`df.groupby('Month').size()`).
- Calculate the mean Ozone level for the days with and without missing values (create a boolean mask from `df['Ozone'].isnull()` — note this is comparing rows where Ozone is missing vs. present, so the "missing" group will need to be reasoned about using a different, complete column, e.g. compare Temp or Wind between the two groups instead).

- Calculate the percentage of missing values in the Ozone column for each month (`df.groupby('Month')['Ozone'].apply(lambda s: s.isnull().mean() * 100)`).

4. Additional Analysis

- Write a custom function to detect outliers using the IQR method (a value is an outlier if it falls below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$).
- Apply the outlier-detection function to the Ozone, Solar.R, Wind, and Temp columns.
- Report the number of outliers detected for each variable.
- Calculate the **skewness** and **kurtosis** for Ozone, Solar.R, Wind, and Temp (`df[cols].skew()`, `df[cols].kurt()`, or `scipy.stats.skew` / `scipy.stats.kurtosis` with `nan_policy='omit'`).
- Report whether the distribution of each variable is symmetric, positively skewed, or negatively skewed, based on the skewness values.
- Identify and report the top three highest Ozone readings and their corresponding Solar.R, Wind, and Temp values (`df.nlargest(3, 'Ozone')`).
- Identify and report the top three lowest Wind readings and their corresponding Ozone, Solar.R, and Temp values (`df.nsmallest(3, 'Wind')`).

5. Comparisons

- Compare the average Temp on days when Ozone is above 60 versus when it is 60 or below.
- Compare the average Solar.R level on days when Wind speed is greater than the mean Wind speed versus when it is less than or equal to the mean.
- Compare the maximum Ozone level across different temperature ranges (below 60°F, 60–80°F, above 80°F) — use `pd.cut()` to create the bins.

6. Data Visualization (New Section)

This section is new relative to the original R assignment, added so the "visualizations to understand the data better" stated in the Objective is explicitly assessed.

- Plot a histogram (with a density curve) for each of Ozone, Solar.R, Wind, and Temp (`seaborn.histplot(..., kde=True)`).
- Plot a boxplot for each of the four variables to visually confirm your outlier counts from Task 4.
- Plot a scatter plot of Ozone vs. Temp, with a fitted regression line (`seaborn.regplot`), and briefly interpret the trend.
- Plot the correlation heatmap generated in Task 2 with annotated coefficient values (`seaborn.heatmap(..., annot=True)`).
- Plot a line chart of mean Ozone level by month (from Task 3) to visualize the seasonal trend.

Submission Guidelines

- Perform this assignment as a Python script (.py) **or** a Jupyter notebook (.ipynb); either write your code in a plain script or work interactively in the notebook.
- Include comments in your code explaining each step (what it computes and why).
- Capture and include the output of every task — either as code-cell outputs in the notebook, or as screenshots pasted below each script block if you used a plain .py file.
- Export your final notebook/script (with outputs) to PDF **or** submit the .ipynb/.py file alongside a PDF of your written report — whichever your instructor's LMS accepts.
- Ensure your report is clear, well organized, and formatted for easy reading, with each of the six sections above clearly labeled.